AI/ML INTERN — DAY 6 REPORT

Name: Praneetha AS
Role: AI/ML Intern
Date: 18/05/2026
Project Folder: Raritone

---

1. Introduction

Artificial Intelligence and Computer Vision systems are widely used in modern applications such as virtual try-on, fashion recommendation systems, and digital human modeling. Day 6 of the internship focuses on advancing previous implementations by improving accuracy, testing robustness, and integrating AI systems into practical workflows.

The tasks performed in this phase aim to move from basic implementation toward real-world application readiness. This includes improving body measurement accuracy, testing pose detection on various inputs, researching cloth segmentation techniques, building a dataset for recommendations, and integrating outputs with backend APIs.

---

2. Task 1 — Improve Body Measurement Accuracy

2.1 Objective

The objective of this task is to improve the accuracy of body measurements derived from pose detection. Instead of only detecting body landmarks, the system now calculates distances between key points to estimate body proportions.

2.2 Concept

MediaPipe provides normalized landmark coordinates. These coordinates are used to compute distances between points such as shoulders, hips, and torso.

2.3 Algorithm

1. Capture video frame
2. Detect pose landmarks
3. Identify key points (shoulders, hips)
4. Apply Euclidean distance formula

5. Display calculated measurement

2.4 Pseudocode

```
START
Capture frame
Detect pose landmarks
IF landmarks detected
   Extract shoulder points
   Calculate distance
   Display measurement
END IF
Repeat
```

2.5 Code Implementation

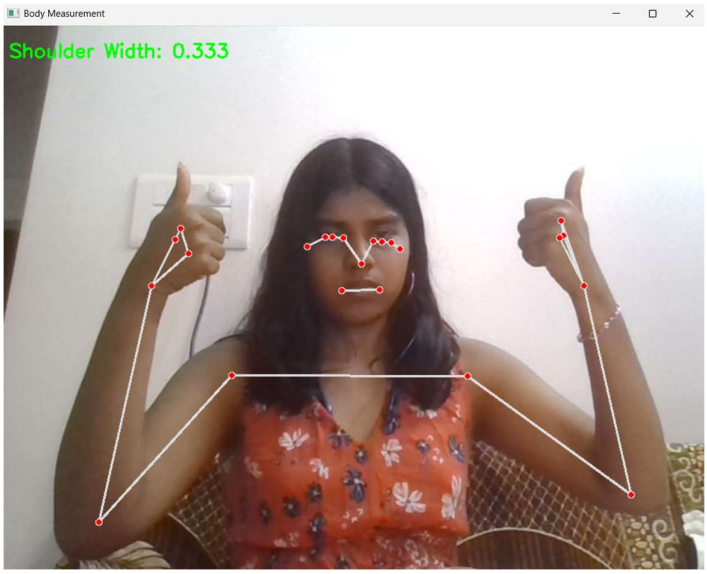
```
import cv2
import mediapipe as mp
import math


mp_pose = mp.solutions.pose
pose = mp_pose.Pose(
   model_complexity=2,
   smooth_landmarks=True,
   min_detection_confidence=0.7,
   min_tracking_confidence=0.7
)
mp_draw = mp.solutions.drawing_utils


def calculate_distance(p1, p2):
   return math.sqrt((p1.x - p2.x)**2 + (p1.y - p2.y)**2)
cap = cv2.VideoCapture(0)
```

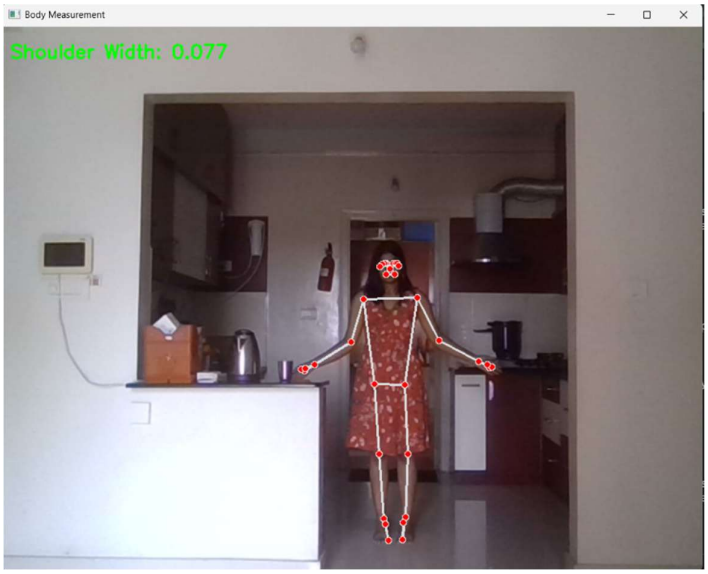
```
while True:
    success, frame = cap.read()
    if not success:
        break

    frame = cv2.resize(frame, (900, 700))
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = pose.process(rgb)

    if results.pose_landmarks:
        lm = results.pose_landmarks.landmark
        left_shoulder = lm[11]
        right_shoulder = lm[12]
        shoulder_width = calculate_distance(left_shoulder, right_shoulder)
        mp_draw.draw_landmarks(
            frame,
            results.pose_landmarks,
            mp_pose.POSE_CONNECTIONS
        )
        cv2.putText(frame,
                    "Shoulder Width: " + str(round(shoulder_width, 3)),
                    (10, 40),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.8,
                    (0, 255, 0),
                    2)
    cv2.imshow("Body Measurement", frame)
```

```
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()
```

2.6 Observations

- Measurement improves when full body is visible
- Accuracy depends on camera distance
- Lighting affects detection quality





2.7 Conclusion

Body measurement accuracy can be improved using landmark-based calculations, enabling better applications in fashion AI systems.

---

3. Task 2 — Test Pose Detection on Images and Videos

3.1 Objective

To evaluate the performance of pose detection across different input formats such as images and videos. This helps in understanding how the model behaves under various conditions and inputs.

3.2 Approach

- **Test pose detection on static images**
- **Test pose detection on video recordings**
- **Compare performance across inputs**

3.3 Algorithm

1. **Load input (image or video)**
2. **Convert frame to RGB format**
3. **Apply pose detection model**
4. **Draw body landmarks and skeleton**
5. **Display output**

3.4 Pseudocode

START
Load input
Process frame
Detect landmarks
Draw skeleton

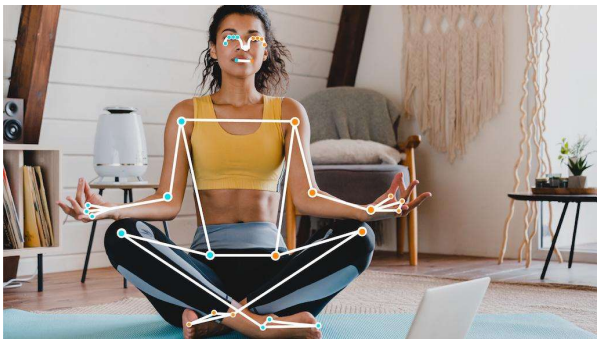
Display result
END

3.5 Code Example

```
import cv2
import mediapipe as mp

mp_pose = mp.solutions.pose
pose = mp_pose.Pose()

img = cv2.imread("test.jpg")
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

results = pose.process(rgb)

if results.pose_landmarks:
   mp.solutions.drawing_utils.draw_landmarks(
      img, results.pose_landmarks, mp_pose.POSE_CONNECTIONS
   )

cv2.imshow("Image Pose", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

3.6 Sample Outputs (Images)

Image Input Pose Detection



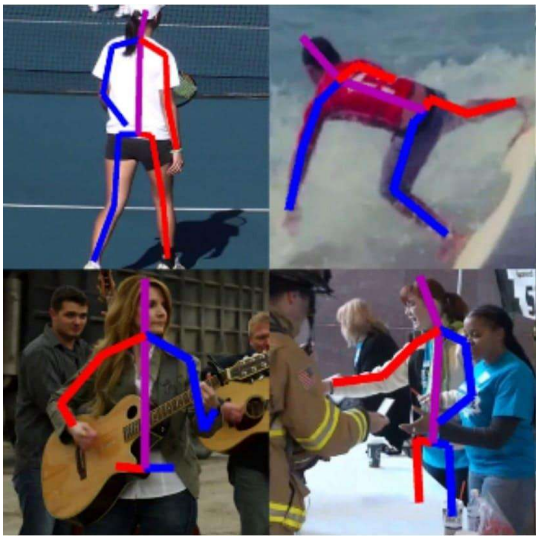
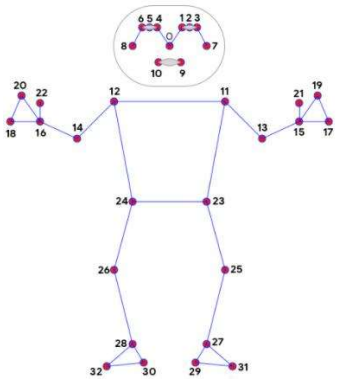


Figure 1: Pose detection on a static image

Video Input Pose Detection



0. nose
1. left_eye_inner
2. left_eye
3. left_eye_outer
4. right_eye_inner
5. right_eye
6. right_eye_outer
7. left_ear
8. right_ear
9. mouth_left
10. mouth_right
11. left_shoulder
12. right_shoulder
13. left_elbow
14. right_elbow
15. left_wrist
16. right_wrist
17. left_pinky
18. right_pinky
19. left_index
20. right_index
21. left_thumb
22. right_thumb
23. left_hip
24. right_hip
25. left_knee
26. right_knee
27. left_ankle
28. right_ankle
29. left_heel
30. right_heel
31. left_foot_index
32. right_foot_index

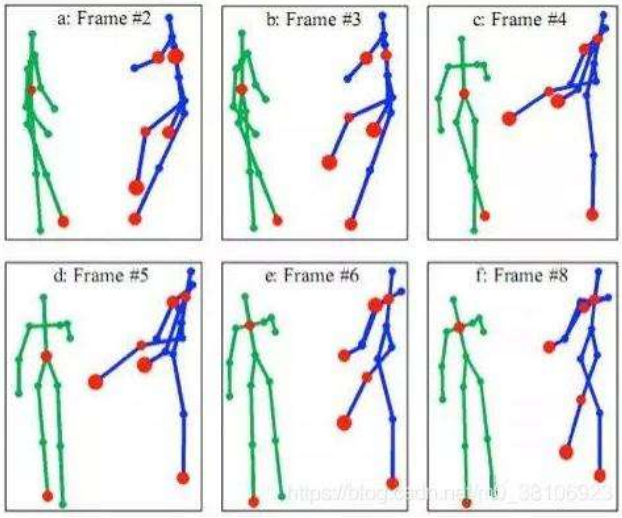


Figure 2: Pose detection on video frames (real-time tracking)

3.7 Observations

- **Video input provides smoother and continuous tracking**
- **Static image detection depends on image clarity and pose**
- **Detection accuracy reduces when body parts are not visible (occlusion)**
- **Better lighting improves detection results**

3.8 Conclusion

Testing pose detection on both images and videos helps in understanding the strengths and limitations of the system. Video-based detection is more stable due to continuous tracking, whereas image-based detection depends heavily on input quality. This testing process ensures robustness and improves system reliability for real-world applications.

---

4. Task 3 — Research Cloth Segmentation AI

4.1 Introduction

Cloth segmentation is a computer vision technique used to separate clothing regions from the human body in images or videos. It plays a crucial role in applications such as virtual try-on systems, fashion analysis, and augmented reality. By identifying clothing pixels separately, AI systems can overlay garments more accurately and realistically.

4.2 Techniques

- Semantic Segmentation — Classifies each pixel into categories such as background, skin, or clothing
- Instance Segmentation — Separates individual objects like shirts, pants, etc.
- Deep Learning Models — Uses neural networks such as U-Net, Mask R-CNN, and DeepLab

---

4.3 Workflow

Input Image → Segmentation Model → Pixel Classification → Output Mask

4.4 Algorithm

1. Input image
2. Detect human region
3. Apply segmentation model
4. Classify each pixel
5. Generate clothing mask

4.5 Pseudocode

START
Input image
Apply segmentation model
Classify pixels
Generate output mask
END

4.6 Sample Outputs (Images)

Cloth Segmentation Output

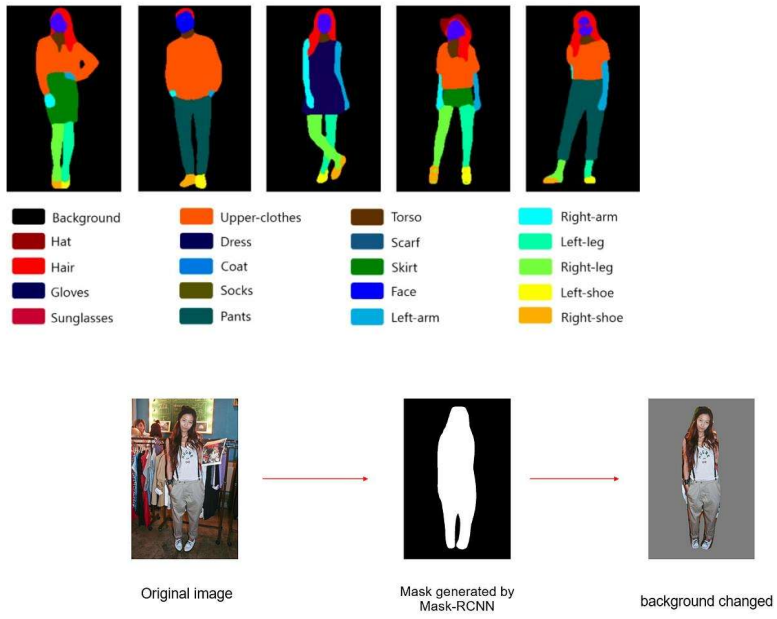


Figure 1: Clothing regions separated from the human body

Pixel-Level Segmentation Visualization

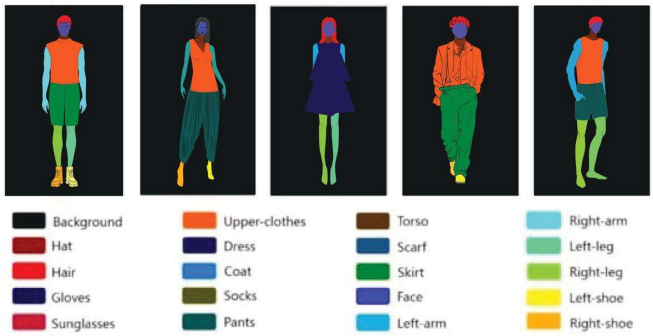


Figure 2: Pixel-level classification of clothing regions

4.7 Observations

- Works best with clear images and simple backgrounds
- Complex backgrounds reduce segmentation accuracy
- High-quality datasets improve model performance
- Deep learning models provide better precision

4.8 Conclusion

Cloth segmentation is a critical component in AI-based fashion systems. It enables accurate separation of clothing from the human body, improving the realism and effectiveness of applications such as virtual try-on and digital styling systems.

---

5. Task 4 — Create Basic Recommendation Dataset

5.1 Objective

To create a dataset that maps body measurements to clothing recommendations.

5.2 Dataset Structure

Size	Color	Outfit
Small	Light	Slim Fit
Medium	Neutral	Regular Fit
Large	Dark	Loose Fit

5.3 Algorithm

1. Input user data
2. Match with dataset
3. Return recommendation

5.4 Pseudocode

START
Input size
Search dataset
Return outfit
END

5.5 Code

```
import pandas as pd

data = pd.read_csv("outfit.csv")
print(data.head())
```

5.6 Observations

- Dataset is simple but scalable
- Can be extended with more features

5.7 Conclusion

Dataset creation is essential for building recommendation systems.

---

6. Task 5 — Integration with Backend APIs

6.1 Objective

To integrate AI outputs with backend systems so that predictions from machine learning models can be accessed through web-based applications and services.

6.2 Approach

- Use Flask API to create a backend server
- Send responses in JSON format
- Connect AI model outputs with API endpoints

6.3 Algorithm

1. Receive request from client
2. Process input data
3. Generate AI output
4. Return response as JSON

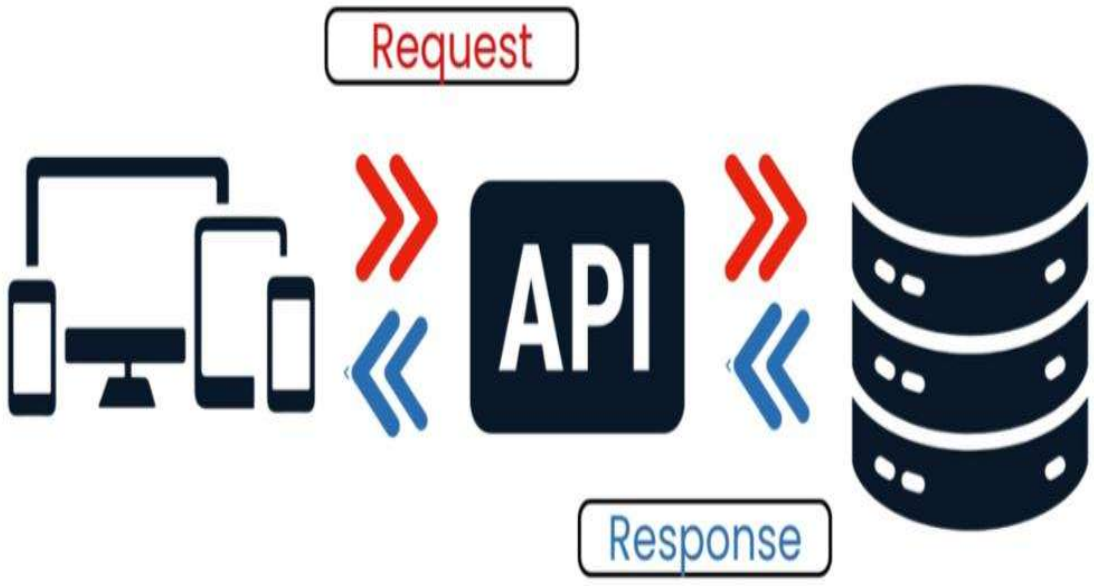
6.4 Pseudocode

START
Receive request
Process input
Generate output
Return JSON response
END

6.5 Code

```
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/predict')
def predict():
    return jsonify({"size": "Medium"})

app.run()
```

6.6 Workflow Visualization



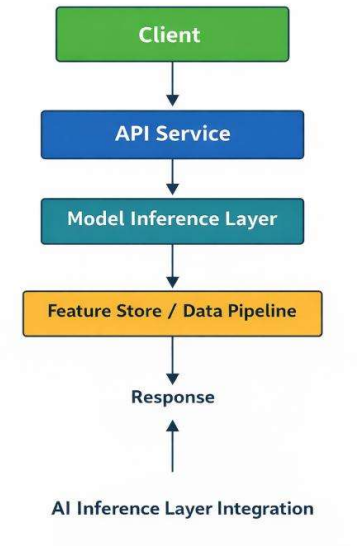


Figure 1: API workflow showing request and response cycle

6.7 Observations

- API integration enables deployment of AI systems
- Real-time responses can be generated
- Backend systems allow scalability
- Integration connects AI with user applications

6.8 Conclusion

Backend integration is essential for deploying AI systems in real-world applications. It allows models to interact with users through web services and supports scalable system design.

---

7. Overall Observations

- Accuracy improves with better lighting and input conditions
- Pose detection requires proper visibility of body parts
- Segmentation accuracy depends on image quality
- Testing across different environments improves system reliability
- Integration is necessary for real-world deployment

---

8. Learning Outcomes

- Understanding of pose detection optimization
- Improved knowledge of body measurement estimation
- Insight into cloth segmentation techniques
- Experience in dataset creation and usage
- Knowledge of backend API integration
- Understanding of real-time AI system workflows

---

9. Final Conclusion

Day 6 focused on advancing AI systems from basic implementation to practical deployment. The tasks improved understanding of computer vision workflows, data processing, and system integration.

These concepts are essential for developing real-world AI applications in fashion technology and beyond.